Day 159 — Line-by-Line Syntax Explanation

MD Mutasim Billah | 52-Week Data Science to ML/AI Roadmap | Week 23, Day 6 reference

Watch: What are Precision@k and Recall@k? (Inside Learning Machines) | Evaluation Measures for Search and Recommender Systems (James Briggs)

`day159_evaluating_recommenders.py`, segment by segment, in the order `main()` uses things. The parts (per_user_metrics, beyond_accuracy, paired_bootstrap, leave_last_out, sampled_eval) are copied unchanged into `day159_exercise.py`; Days 154, 155 and 158's parts are imported from their exercise files. Code boxes are cut from the real script by program (inline comments removed). Purple **MATH** boxes come before code that implements a formula. A glossary is at the end.

MATH BOX LEGEND

Symbols: S = scores, users × films · seen = already watched · gain = worth of each film to find (0 = not to find) · top = the lists · pop = watchers per film · d = per-user differences

Segment 0 — Setup

Imports and the settings carried over

CODE

```
HERE = os.path.dirname(os.path.abspath(__file__))
sys.path.insert(0, HERE)
with contextlib.redirect_stdout(io.StringIO()):
    import day154_exercise as d154
    import day155_exercise as d155
    import day158_exercise as d158
IALS = dict(k=50, lam=30, alpha=3)
STAR_ALPHA = 0.6
SVD_K = 20
K = 10
```

- `import day158_exercise as d158` — Day 158's parts: implicit_als and cooccurrence.
- `IALS = dict(k=50, lam=30, alpha=3)` — the settings Day 158 chose on validation, reused rather than re-tuned on test.

Segment 1 — The parts

per_user_metrics: five measures at once

MATH

$$AP = \Sigma\ precision@j \cdot hit_j / min(n, k); \quad NDCG = \Sigma\ gain_j / log_2(j + 1) / ideal$$

CODE

```
def per_user_metrics(S, seen, gain, k=10):
    S = np.where(seen, -np.inf, S); held = gain > 0
    who = np.flatnonzero(held.sum(1) > 0)
    order = np.argsort(-S, axis=1, kind="stable")
    top = order[:, :k]; hit = np.take_along_axis(held, top, 1)
    disc = 1.0 / np.log2(np.arange(2, k + 2))
    dcg = (np.take_along_axis(gain, top, 1) * disc).sum(1)
    idcg = (-np.sort(-gain, axis=1)[:, :k] * disc).sum(1)
    nrel = held.sum(1)
    ap = ((np.cumsum(hit, 1) / np.arange(1, k + 1)) * hit).sum(1) / np.maximum(np.minimum(nrel,
        k), 1)
    pos = np.empty_like(order); np.put_along_axis(pos, order, np.arange(S.shape[1])[None,
        :].repeat(len(S), 0), 1)
    rr = 1.0 / (np.where(held, pos, S.shape[1]).min(1) + 1)
    return dict(precision=hit.sum(1)[who] / k, recall=(hit.sum(1) / np.maximum(nrel, 1))[who],
        map=ap[who],
    ndcg=(dcg / np.maximum(idcg, 1e-12))[who], mrr=rr[who]), top[who]
```

- `np.take_along_axis(held, top, 1)` — for each user, True where the film at that place is one to find.
- `1.0 / np.log2(np.arange(2, k + 2))` — the discount for places 1 to k: 1/$\log_2$2, 1/$\log_2$3, ...
- `-np.sort(-gain, axis=1)[:, :k]` — each row sorted from largest gain: the best possible list.
- `np.cumsum(hit, 1) / np.arange(1, k + 1)` — precision at every place 1 to k, in one step.
- `np.put_along_axis(pos, order, ..., 1)` — the inverse of the ranking: for each film, its place.
- `np.where(held, pos, S.shape[1]).min(1)` — the place of the best-ranked film to find (films not to find count as last).
- `dict(precision=..., ...)` — one array per measure, only for users with something to find.

beyond_accuracy and paired_bootstrap

MATH

$$novelty = mean\ -log_2(watchers / users); \quad personalisation = 1 - mean\ overlap\ of\ two\ lists$$

CODE

```
def beyond_accuracy(top, pop, nu):
    L = np.zeros((len(top), len(pop))); L[np.arange(len(top))[:, None], top] = 1.0
    ov = L @ L.T / top.shape[1]; n = len(top)
    return dict(coverage=len(np.unique(top)) / len(pop),
    median_pop=float(np.median(pop[top])),
    novelty=float(np.mean(-np.log2(np.maximum(pop[top], 1) / nu))),
    personal=1.0 - (ov.sum() - np.trace(ov)) / (n * (n - 1)))
def paired_bootstrap(a, b, B=2000, seed=0):
    d = a - b; rng = np.random.RandomState(seed)
    m = d[rng.randint(0, len(d), (B, len(d)))].mean(1)
    return d.mean(), np.percentile(m, [2.5, 97.5]), (m <= 0).mean(), m
```

- `L[np.arange(len(top))[:, None], top] = 1.0` — a column of row numbers against the lists: marks every listed film in one step.
- `L @ L.T / top.shape[1]` — films in common between every pair of lists, as a share of the list length.
- `ov.sum() - np.trace(ov)` — all pairs minus each list with itself (the diagonal).

- `rng.randint(0, len(d), (B, len(d)))` — B rows of random user numbers, repeats allowed: B resamples at once.
- `np.percentile(m, [2.5, 97.5])` — the middle 95% of the resampled means.

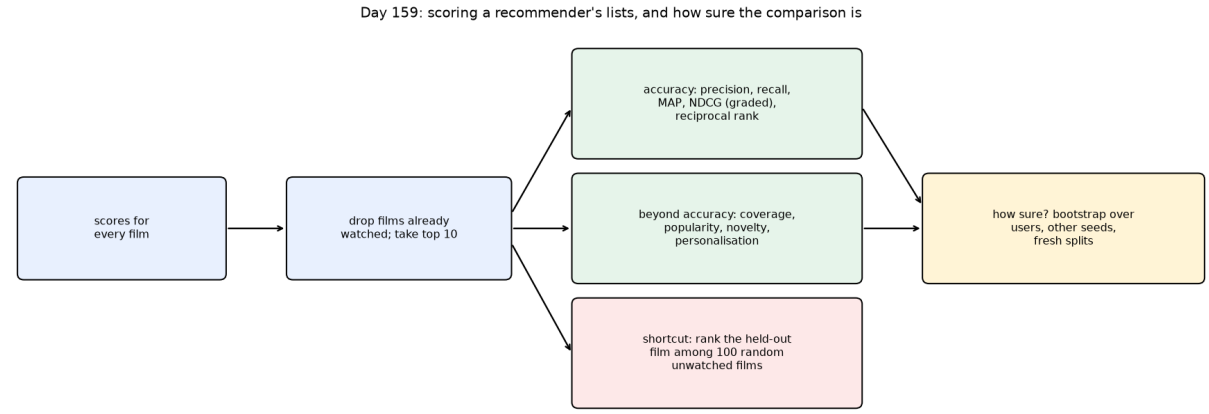
leave_last_out and sampled_eval

CODE

```
def leave_last_out(u, t):
    order = np.lexsort((np.arange(len(t)), t, u))
    last = order[np.r_[np.flatnonzero(np.diff(u[order])), len(u) - 1]]
    keep = np.ones(len(u), bool); keep[last] = False
    return np.flatnonzero(keep), last
def sampled_eval(S, seen, item, n_neg=100, k=10, seed=0):
    rng = np.random.RandomState(seed); hr, nd = [], []
    for a in range(len(item)):
        cand = np.flatnonzero(~seen[a]); cand = cand[cand != item[a]]
        neg = rng.choice(cand, n_neg, replace=False)
        rank = (S[a, neg] >= S[a, item[a]]).sum()
        hr.append(rank < k); nd.append(1.0 / np.log2(rank + 2) if rank < k else 0.0)
    return np.mean(hr), np.mean(nd)
```

- `np.lexsort((np.arange(len(t)), t, u))` — sorts by the LAST key first: user, then time, then position to break ties.
- `np.flatnonzero(np.diff(u[order]))` — where the user changes in the sorted order: the end of each user's block.
- `np.r_[..., len(u) - 1]` — adds the very last row, the end of the final block.
- `rng.choice(cand, n_neg, replace=False)` — 100 different unwatched films.
- `(S[a, neg] >= S[a, item[a]]).sum()` — how many sampled films score at least as high: the target's place, counted from 0.

SYSTEM DIAGRAM



Scoring lists, and how sure a comparison is.

Segment 2 — The experiments

models and held: every model from one set of rows

CODE

```
M = np.zeros((nu, ni), bool); M[u[idx], i[idx]] = True; B = M.astype(float)
R, Mr = d154.to_matrix(u[idx], i[idx], r[idx], nu, ni)
Fe, _ = d155.als_mf(R, Mr, 100, 0.125)
X, Y, _ = d158.implicit_als(np.where(M, 1.0 + IALS["alpha"], 0.0), IALS["k"], IALS["lam"],
    seed=seed)
Xs, Ys, _ = d158.implicit_als(np.where(M, 1.0 + STAR_ALPHA * R, 0.0), IALS["k"], IALS["lam"],
    seed=seed)
return M, {"popularity": np.broadcast_to(B.sum(0), M.shape),
    "random": np.random.RandomState(seed).rand(nu, ni),
    "co-occurrence": B @ d158.cooccurrence(B),
    "PureSVD": d155.low_rank(B, SVD_K),
    "explicit ALS": Fe["mu"] + Fe["bu"][:, None] + Fe["bi"][None, :] + Fe["P"] @ Fe["Q"].T,
    "implicit ALS": X @ Y.T,
    "implicit ALS, stars": Xs @ Ys.T}
held(u, i, r, idx, nu, ni, graded):
G = np.zeros((nu, ni)); G[u[idx], i[idx]] = 2.0 ** r[idx] - 1 if graded else 1.0
return G
```

- `np.where(M, 1.0 + STAR_ALPHA * R, 0.0)` — confidence from the stars where watched, for the star-weighted model.
- `{"popularity": ..., ...}` — a dictionary of score tables, so every later loop treats the models the same way.
- `2.0 ** r[idx] - 1 if graded else 1.0` — a conditional expression: graded gains, or 1 for every film to find.

demo_models and demo_sure: measures, orders and resamples

CODE

```
for c, mn in enumerate(("precision", "recall", "MAP", "NDCG", "graded NDCG", "MRR")):
    print(f"  order by {mn:>11}: " + " > ".join(names[q] for q in np.argsort([-res[x][c] for x in
        names], kind="stable")))
for s in (10, 11, 12, 13, 14):
    tr2, te2 = d154.split_ratings(len(r), 0.2, seed=s)
    M2 = np.zeros((nu, ni), bool); M2[u[tr2], i[tr2]] = True
    R2, _ = d154.to_matrix(u[tr2], i[tr2], r[tr2], nu, ni); G2 = held(u, i, r, te2, nu, ni, True)
    X, Y, _ = d158.implicit_als(np.where(M2, 1.0 + IALS["alpha"], 0.0), IALS["k"], IALS["lam"])
    Xs, Ys, _ = d158.implicit_als(np.where(M2, 1.0 + STAR_ALPHA * R2, 0.0), IALS["k"],
        IALS["lam"])
    nb, ns = per_user_metrics(X @ Y.T, M2, G2)[0]["ndcg"].mean(), per_user_metrics(Xs @ Ys.T, M2,
        G2)[0]["ndcg"].mean()
    diffs.append(ns - nb)
```

- `np.argsort([-res[x][c] for x in names], kind="stable")` — the models from best to worst on measure c.
- `" > ".join(...)` — the names joined into one line, with > between them.
- `d154.split_ratings(len(r), 0.2, seed=s)` — a fresh 80/20 split for each seed; both models are refitted on it.

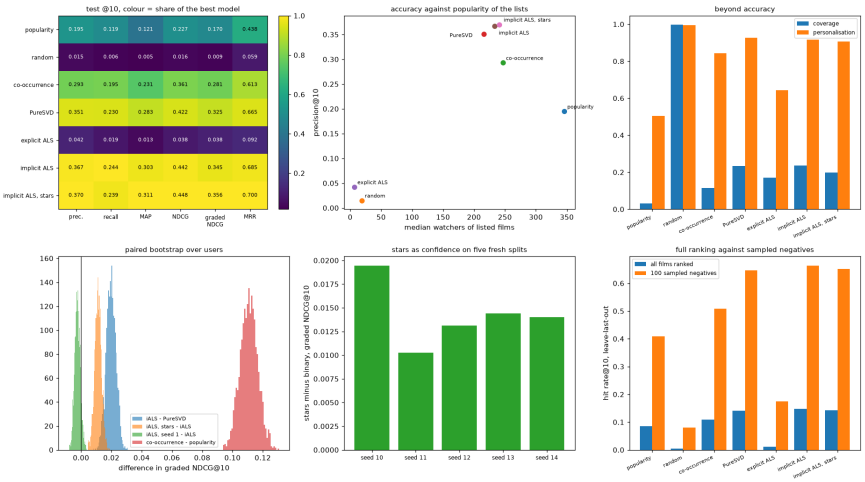
demo_sampled: the shortcut, two ways

CODE

```
item = np.zeros(nu, int); item[u[tel]] = i[tel]
G = np.zeros((nu, ni)); G[np.arange(nu), item] = 1.0
other = [sampled_eval(np.random.RandomState(s).rand(nu, ni), M, item)[0] for s in (1, 2, 3, 4, 5)]
```

- `G[np.arange(nu), item] = 1.0` — row a, column item[a]: exactly one film to find per user.
- `[sampled_eval(np.random.RandomState(s).rand(nu, ni), ...)[0] for s in ...]` — five fresh tables of random scores, the check on 10/101.

REAL GRAPH



Real graph from the script.

Quick Syntax Glossary — new or reinforced constructs

- `np.take_along_axis(a, idx, 1)` — pick per row by index.
- `np.put_along_axis(a, idx, v, 1)` — write per row by index.
- `np.cumsum(x, 1)` — running totals along each row.
- `np.lexsort((c, b, a))` — sort by a, then b, then c.
- `np.diff(x)` — differences of neighbours.
- `np.percentile(m, [2.5, 97.5])` — an interval from samples.
- `np.trace(A)` — sum of the diagonal.
- `x if cond else y` — a conditional expression.

End of Day 159 syntax reference. For the experiments and what they showed, see the Day 159 complete guide.